

Baton

One account deck that keeps its history as ownership changes.

[Transfer evidence](#) / [Python checks](#) / [Skill assessment](#) / [Saved history](#) / [Account deck](#)

Why I built it

When an account changes hands, the next team often has to reconstruct what the customer wanted and what is still owed. I built Baton to keep that context in one deck. The front shows the latest owner and outstanding work; dated sections preserve earlier handoffs. The intended benefit is less preparation and fewer dropped commitments. I have not measured time savings or customer impact.

What I assumed

A person supplies each transfer, its date, and both internal owners and teams. A current CRM owner cannot establish the previous owner or when a change happened. Internal employees and customer contacts are different roles. For the initial sales handoff, missing sponsorship, timeline, or success criteria blocks readiness. Later transfers use current responsibilities and outstanding commitments instead. These are prototype policies to agree with the receiving teams.

How I split the work

The initial sales check reads five Salesforce-style CSVs: Account, Opportunity, Contact, OpportunityContactRole, and SalesNotes. They provide the external data source. Python validates those records and protects the handoff history from duplicate events, conflicting updates, chronology errors, and owner discontinuity. The skill assesses meaning, cites evidence, and records follow-up actions. The renderer rebuilds the PowerPoint from saved history, preserving earlier content. Python helpers use the standard library; deck generation also needs Node.js and the presentation library supplied in the demo's Codex environment.

The example I would show

Terrapin Touring Co. moves from Sales to Implementation on September 16, 2026 with a Ready assessment. A simulated December 1 transfer gives Althea Cassidy in Customer Success the account, with Needs Review because training and QBR dates remain unconfirmed. The September section keeps its original assessment. An open December 18 investigation-time check carries forward even though the second event omits it. Ruben in Implementation remains responsible for that action until evidence supports a change. A new account owner does not silently inherit every action.

Where AI helped, and needed checking

I used AI to draft the skill, code, fictional cases, and supporting materials, then asked for review and failure tests. The initial sales validator could select the wrong deal, accept TBD, and hide broken contact links. Those findings drove fixes. All 56 tests pass: 27 sales tests and 29 history tests. Five sales fixtures and a separate two-export agent review checked the distinction between filled fields and reliable evidence. History tests cover saved records and open actions. These checks test specific behavior, not broad model accuracy or the truth of customer statements.

What I would change for production

I would connect to actual ownership history with limited access, define each team's handoff requirements, and add monitored runs with safe retries. Shared use needs concurrent-write handling and a reviewed correction process; this local prototype expects one writer. Customer notes need access controls and retention rules. At scale I would fetch only relevant changes and handle API limits. I would evaluate labeled real handoffs for missed gaps and false alarms. There is no automatic CRM trigger or writeback today, and manual slide edits do not update saved history.